Estruturas de Dados Básicas II

Prof. Dr. Matheus da Silva Menezes



UFRN
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Começando a Unidade 02.

Estruturas de Dados Básicas II

Árvores
Árvore Binária de Busca

Definição

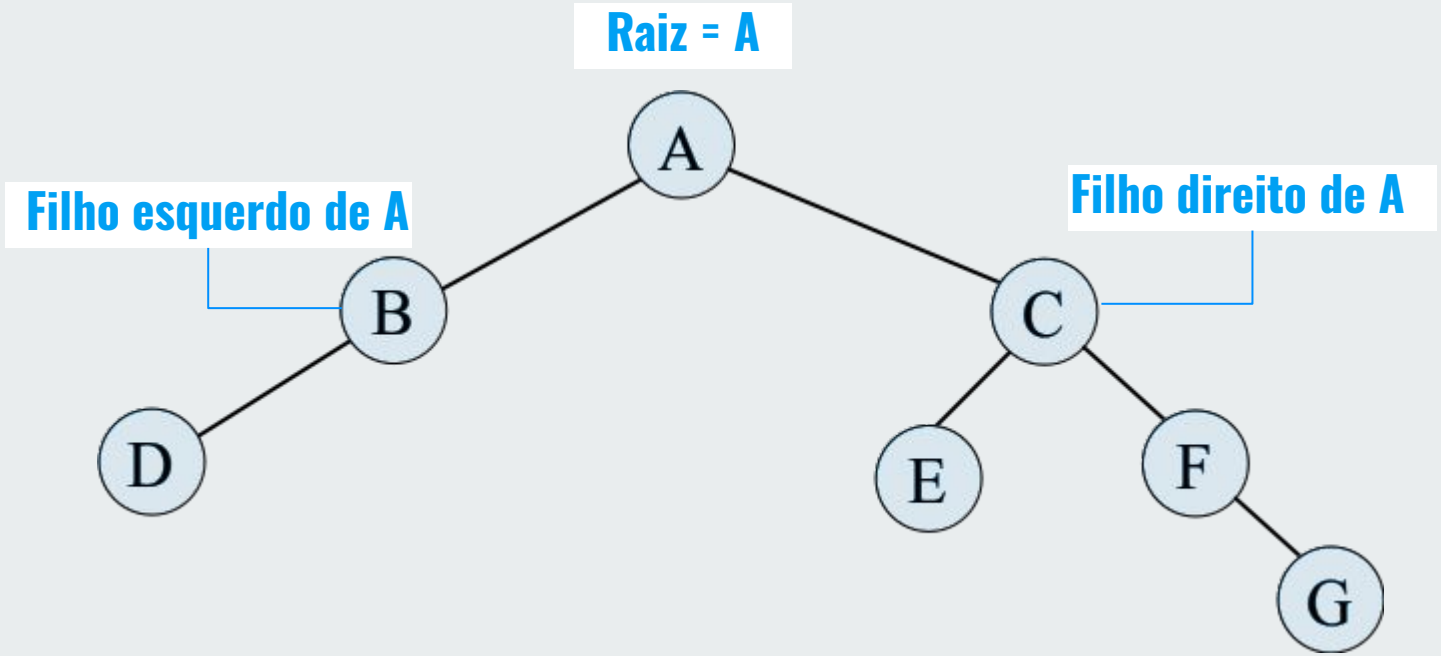
Uma árvore binária, `T`, é um conjunto finito de elementos chamados vértices (ou nós), tal que:

i) `T` = ∅ (a árvore é dita vazia)

ou

ii) Existe um nó especial, `r` dito raiz da árvore `T` e os nós restantes são divididos em dois conjuntos disjuntos, a **subárvore** *esquerda* e *direita* de `r`, cada qual uma árvore binária.

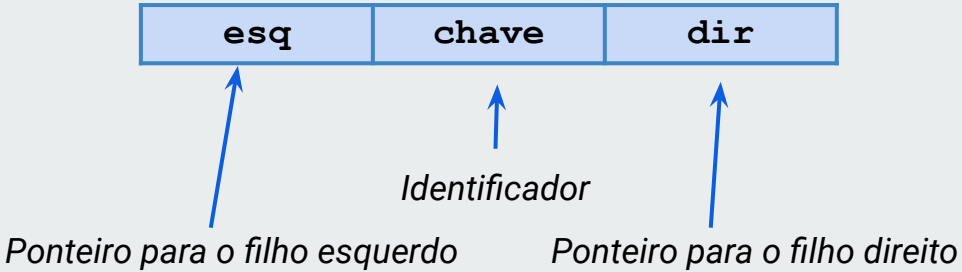



Raiz = A
A
Filho esquerdo de A
Filho direito de A
B
C
D
E
F
G

Uma representação para nó de uma árvore binária

Exemplo C

```
struct nodo{
  int chave;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
}
```




```
typedef no *pont_no;
```

Observação: O campo chave será usado como chave e conteúdo

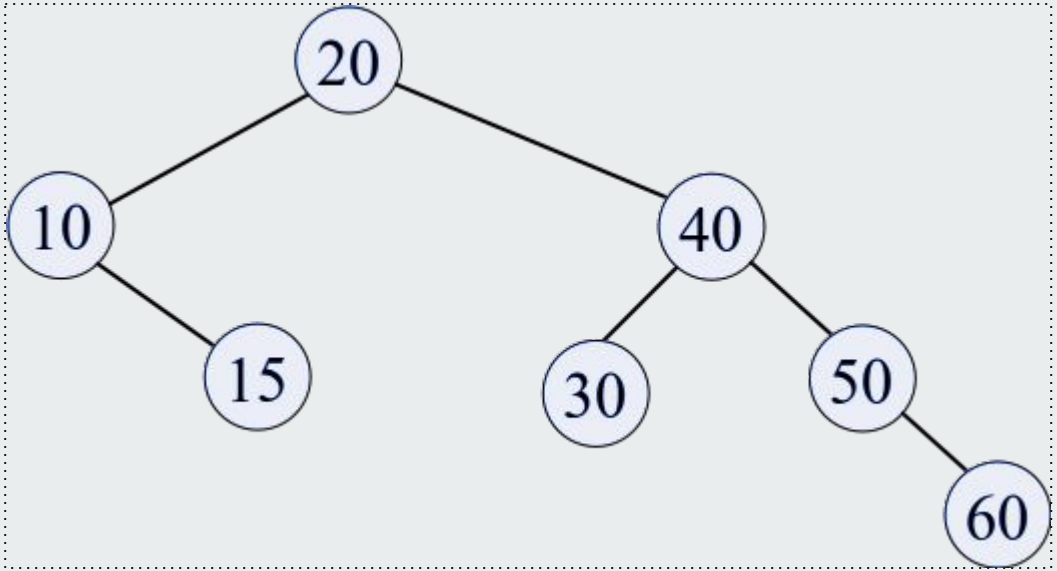
Definição

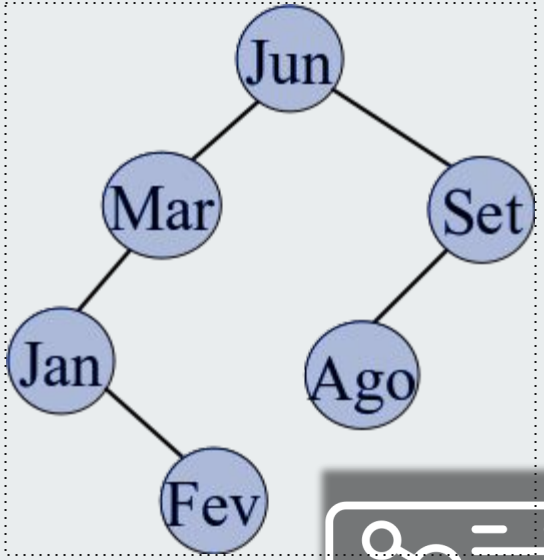
Uma árvore binária de busca é uma árvore binária tal que:

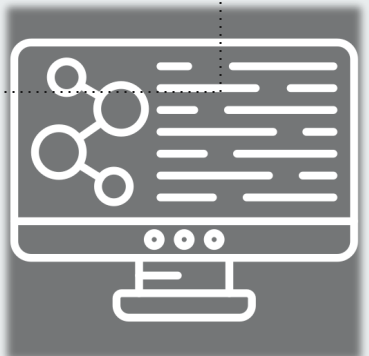
- *A raiz possui uma chave*
- *As chaves dos nós da subárvore **esquerda** da raiz são **menores** que a chave da raiz.*
- *As chaves dos nós da subárvore **direita** da raiz são **maiores** que a chave da raiz.*
- *As subárvores esquerda e direita são árvores binárias de busca*



Exemplos

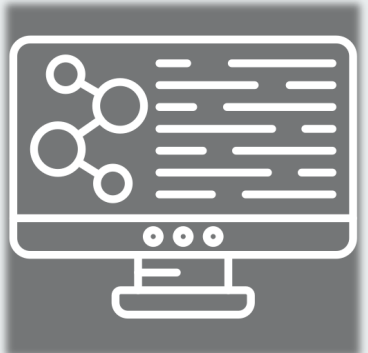






Operações Básicas

1. **BUSCA**
2. **INSERÇÃO**
3. **REMOÇÃO**



Operações Básicas

1. **BUSCA**
2. **INSERÇÃO**
3. **REMOÇÃO**

BUSCA

Considere $S = \{s_1, \ldots, s_n\}$ um conjunto de n chaves tais que:

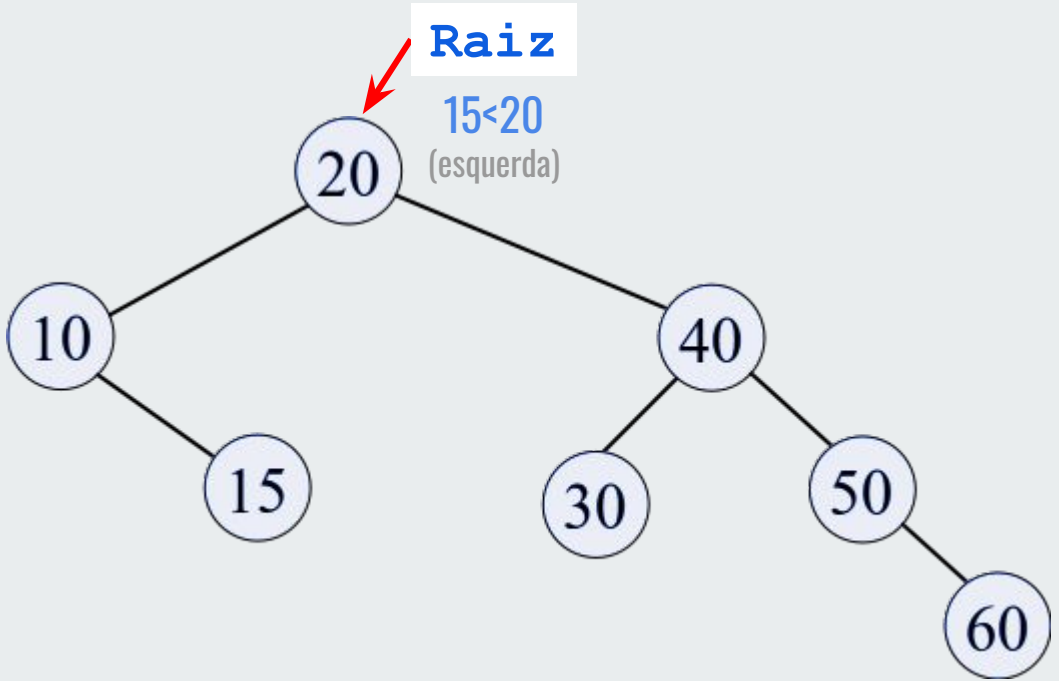
$s_1 < \ldots < s_n$

Dado um valor x (chave de busca), verificar se $x \in S$.

Caso $x == s_i$, $s_i \in S$, retornar i.

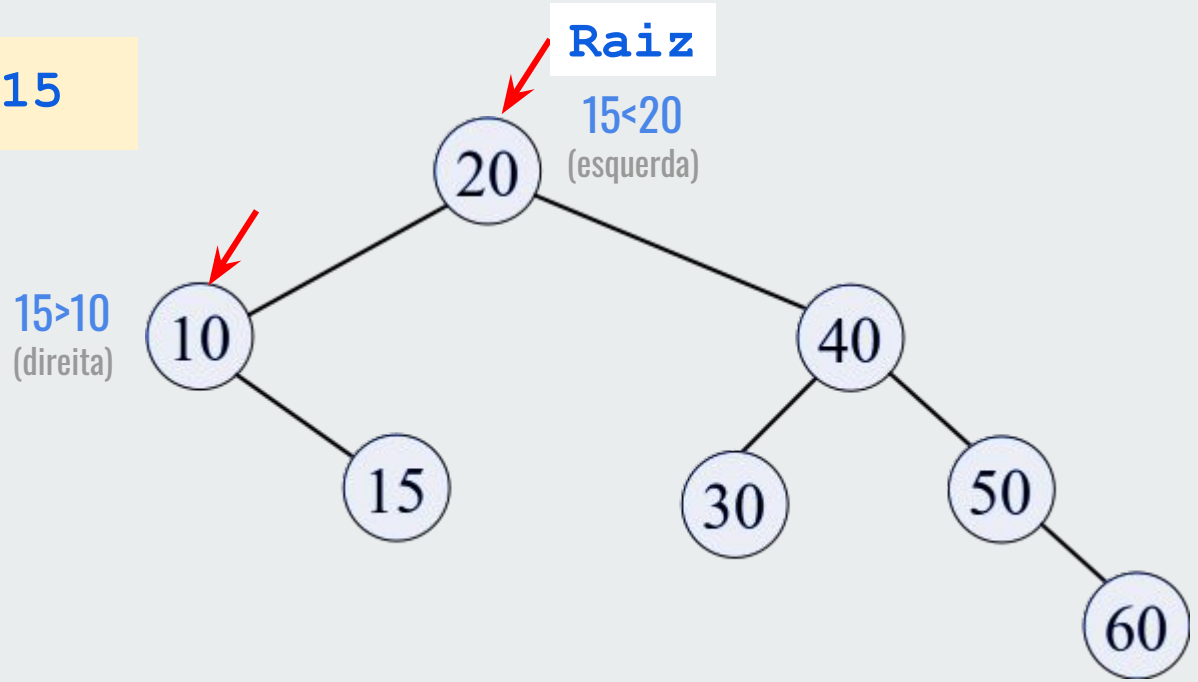
Exemplo

Chave de Busca: **15**



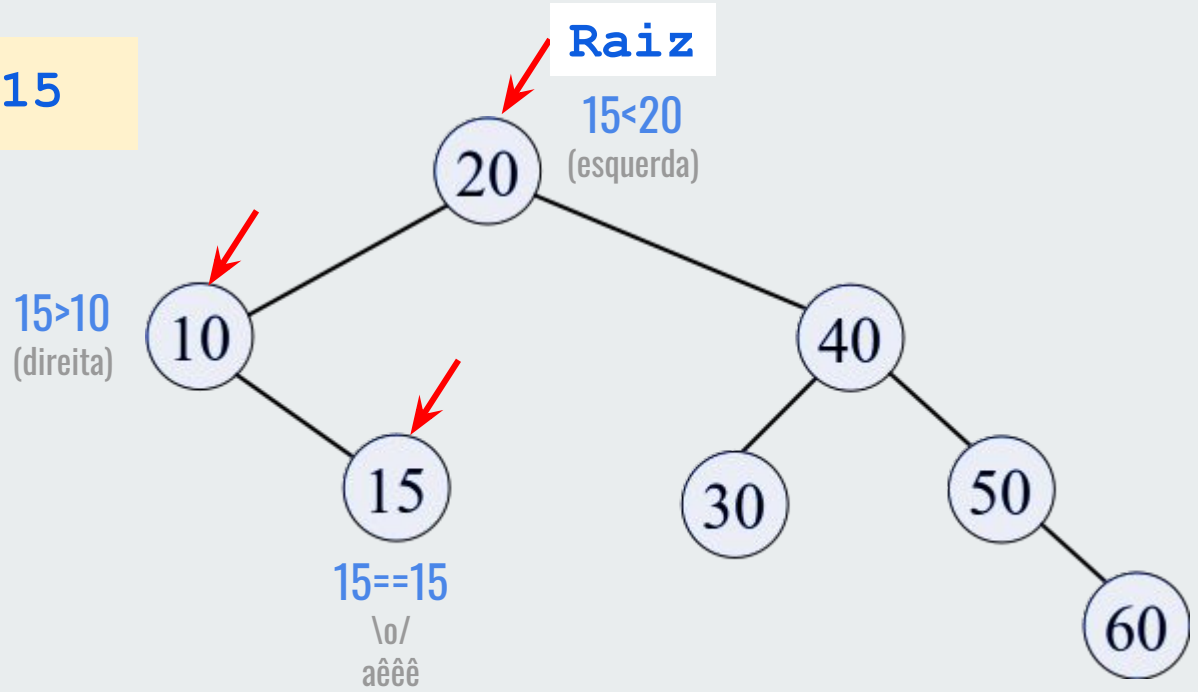
Exemplo

Chave de Busca: **15**



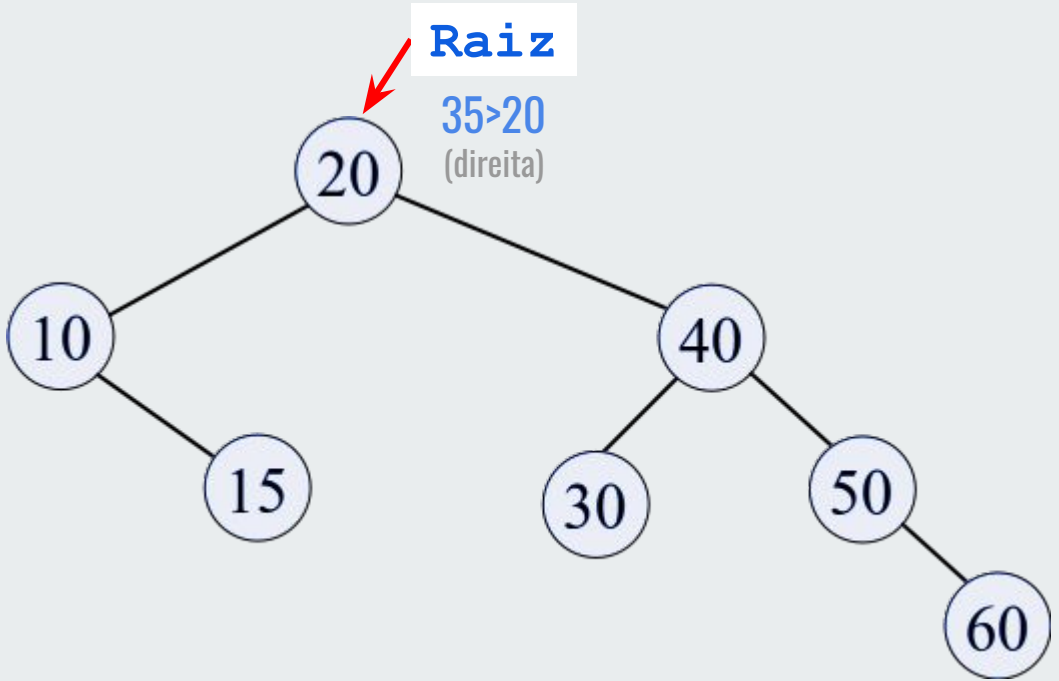
Exemplo

Chave de Busca: **15**



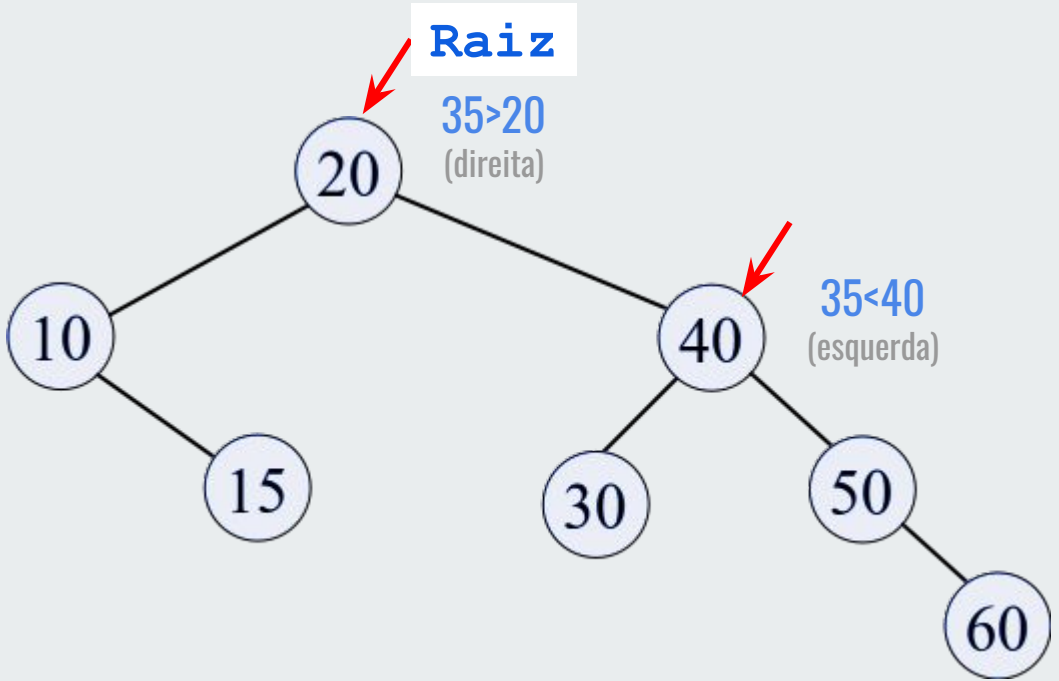
Outro Exemplo

Chave de Busca: **35**



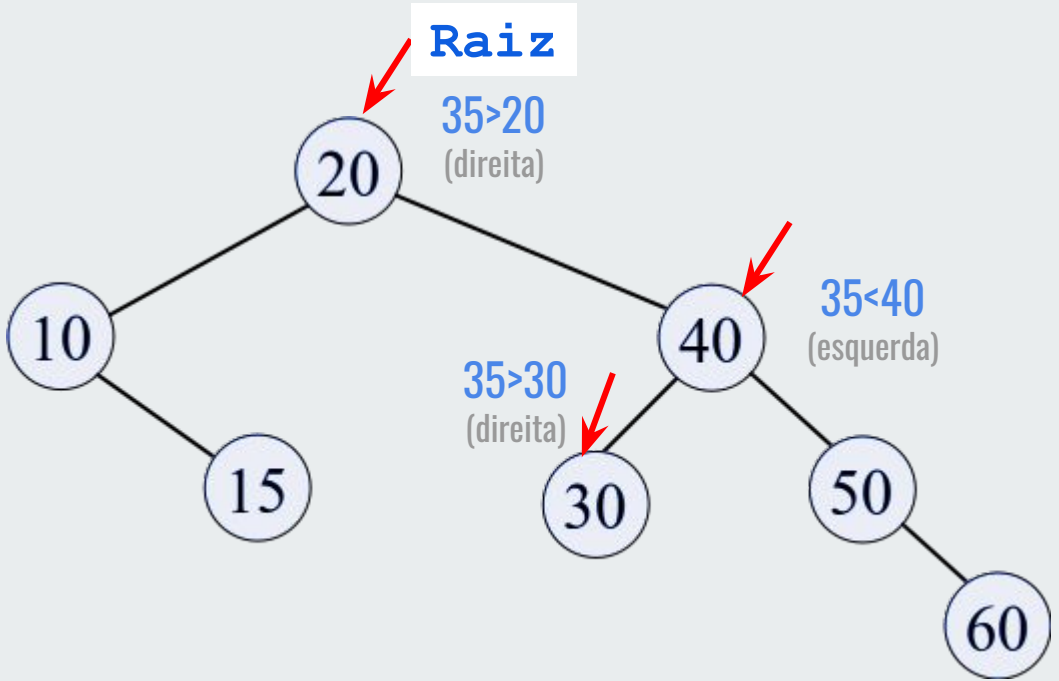
Outro Exemplo

Chave de Busca: **35**



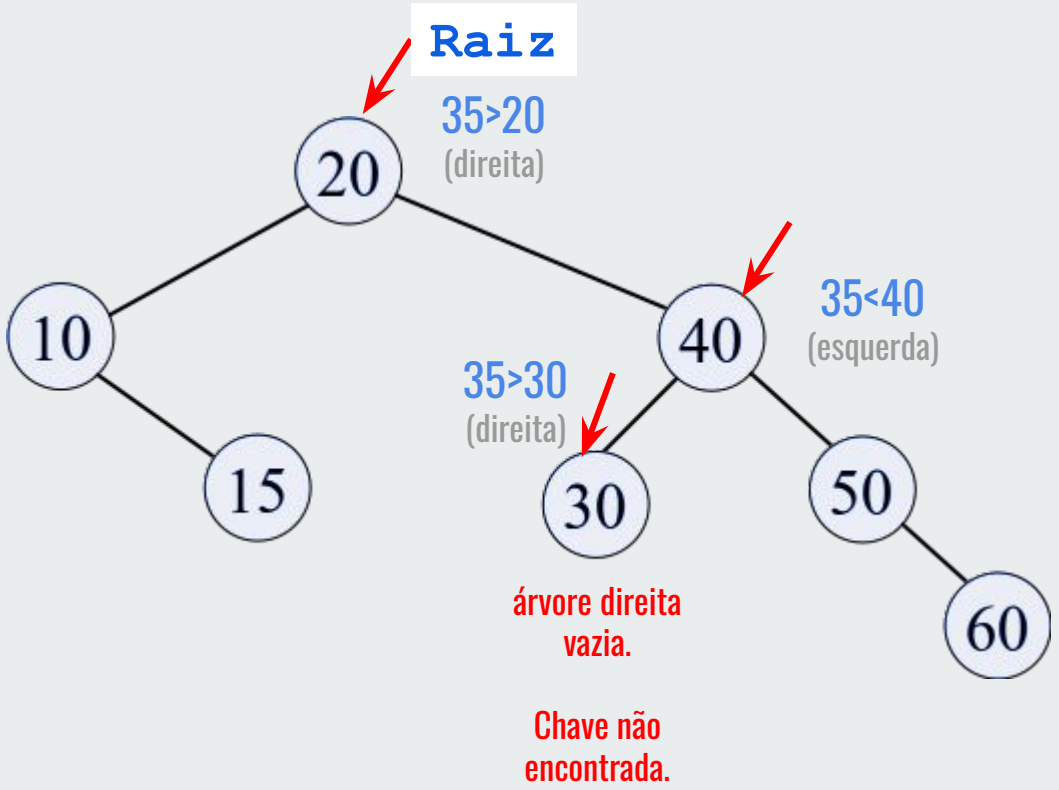
Outro Exemplo

Chave de Busca: **35**



Outro Exemplo

Chave de Busca: **35**



Pseudocódigo

```
struct nodo{
  int chave;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
}
typedef no *pont_no;
```

Pseudocódigo

```
void busca(pont_no raiz, int chave){
   if (raiz== NULL||raiz->chave == chave))
      return raiz;
   if (chave < raiz->chave) // Busca na subárvore esquerda
      return busca (raiz->esq, chave);
   if (chave > raiz->chave) // Busca na subárvore direita
      return busca (raiz->dir, chave);
}
```

Busca Recursiva

Pseudocódigo

```
struct nodo{
  int chave;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
}
typedef no *pont_no;
```

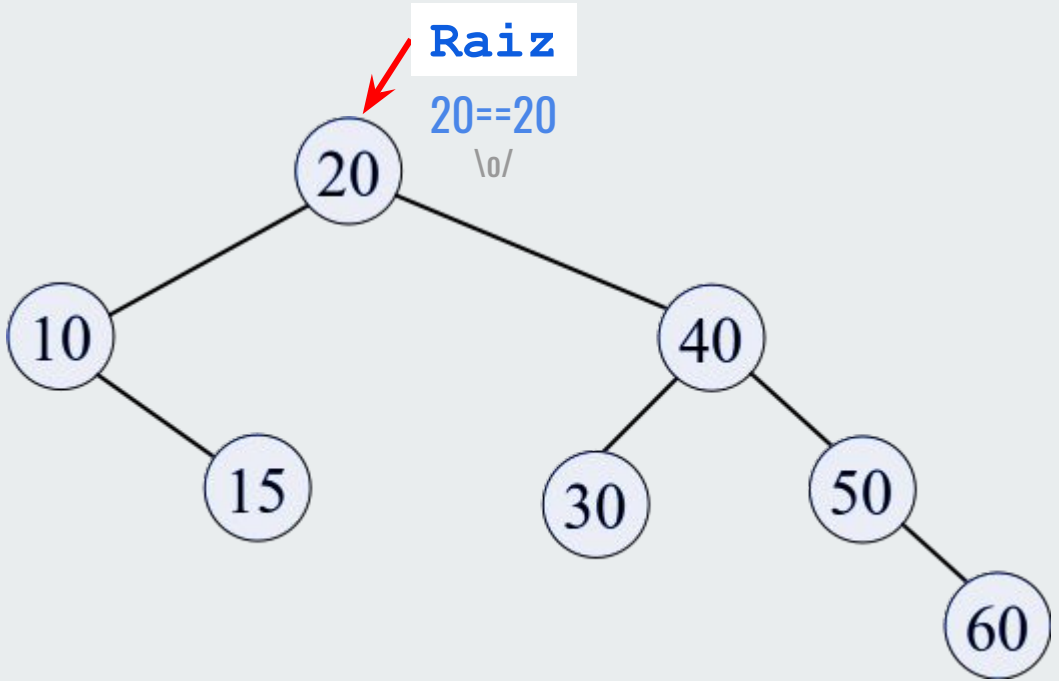
Pseudocódigo

```
void busca2(pont_no raiz, int chave){
  while (raiz!=NULL && raiz->chave != chave){
    if (chave < raiz->chave) // Busca na subárvore esquerda
      raiz = raiz->esq;
    else // Busca na subárvore direita
      raiz = raiz->dir;
  }
  return raiz;
}
```

Busca Iterativa

Complexidade da Busca

Chave de Busca: **20**



Melhor Caso: Chave está na raiz

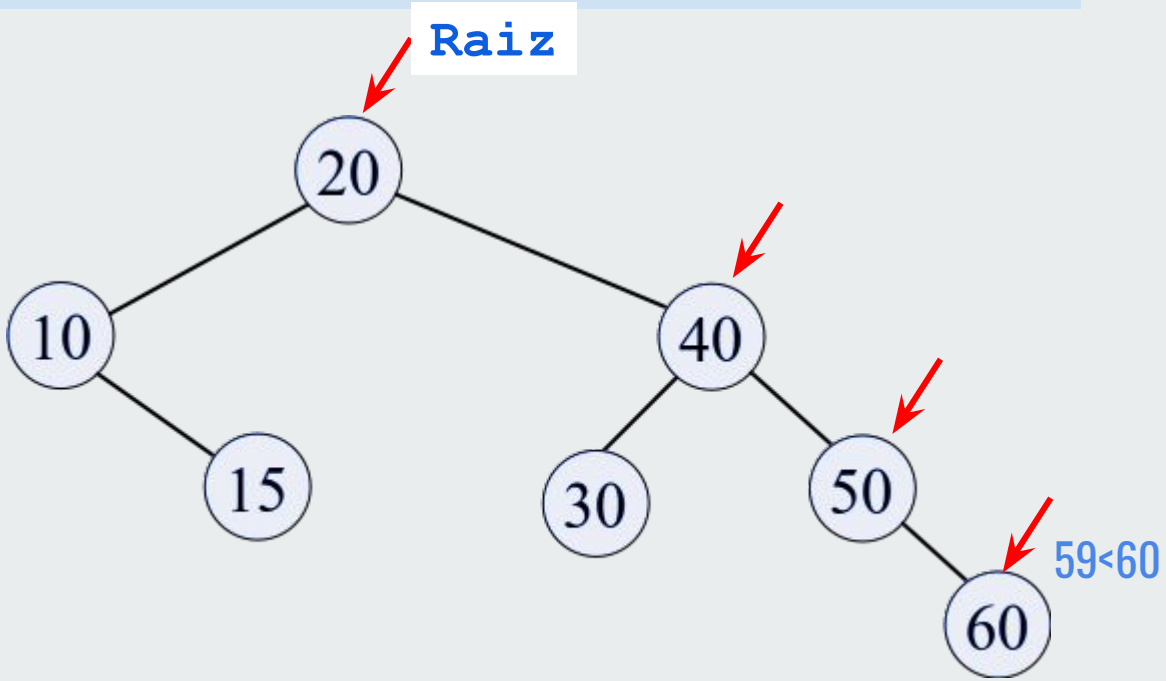
Complexidade da Busca

Pior caso?



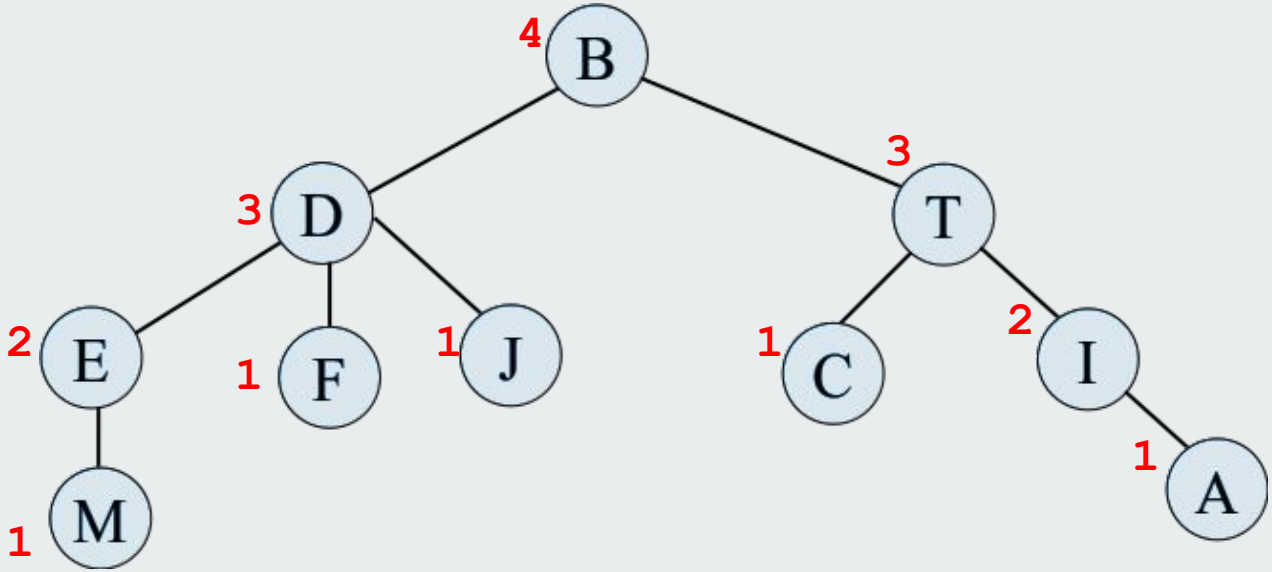
Complexidade da Busca

Chave de Busca: **59**



Pior caso: Chave não encontrada ou encontrada no maior caminho entre a raiz e uma folha = **`h(T)`**

Altura do nó `v` é o número de nós no caminho entre `v` e seu ***descendente mais distante***.



A ***altura da árvore***, `h(T)`, é dada pela ***altura da raiz*** (ou pelo nível máximo de seus nós)

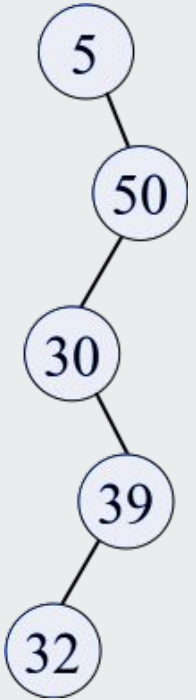
1. Toda árvore com número de nós maior que 1 $(n > 1)$ possui no mínimo 1 e no máximo $(n - 1)$ folhas.

2. O número de subárvores vazias de uma árvore binária com n nós é $(n + 1)$.

3. Considere T uma **árvore binária completa** com $n > 0$ nós. Então T possui altura h mínima. Além disso, $h = 1 + \lfloor \log n \rfloor$.

4. Seja T uma árvore binária completa com n nós e altura h. Então, $2^{h-1} \leq n \leq 2^h - 1$

Árvore zig-zag



$h(T) = O(n)$

Árvore Binária Completa

Se `v` é um nó com uma subárvore vazia, então `v` está no último ou no penúltimo nível de `T`.

A complexidade da busca depende da altura da árvore.

Sendo assim, é interessante construir a árvore com a menor altura possível.

Como visto anteriormente, a árvore que possui altura mínima para um conjunto de `n` chaves é a **árvore binária completa**. Nesse caso, a complexidade do algoritmo é `O(log n)`.

Operações Básicas

1. BUSCA
2. **INSERÇÃO**
3. REMOÇÃO

INSERÇÃO

O novo nó é inserido no lugar de uma subárvore vazia.

Pseudocódigo

```
struct nodo{
  int chave;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
}
typedef no *pont_no;
```

Observação: se chave é igual a algum valor já existente, o código ignora a inserção (não faz nada)

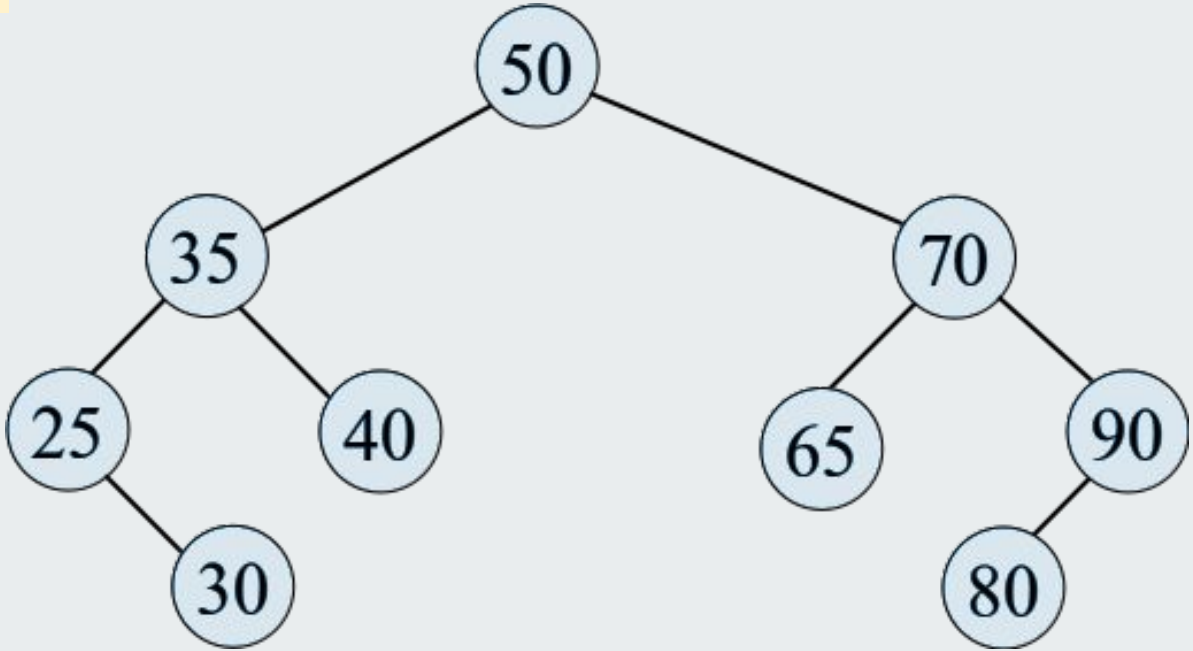
Pseudocódigo

```
pont_no inserir(pont_no raiz, int chave){
  // Se a árvore está vazia, cria um novo nó e retorna
  if (raiz == NULL) {
    pont_no novo = (pont_no)malloc(sizeof(no));
    novo->chave = chave;
    novo->esq = novo->dir = NULL;
    return novo;
  }
  // Se a chave é menor, insere na subárvore esquerda
  if (chave < raiz->chave)
    raiz->esq = inserir(raiz->esq, chave);
  // Se a chave é maior, insere na subárvore direita
  else if (chave > raiz->chave)
     raiz->dir = inserir(raiz->dir, chave);
return raiz; // Retorna a raiz (não alterada)
}
```

Inserção: O novo nó é inserido no lugar de uma subárvore vazia.

Inserção: O novo nó é inserido no lugar de uma subárvore vazia.

Inserir a chave: **42**

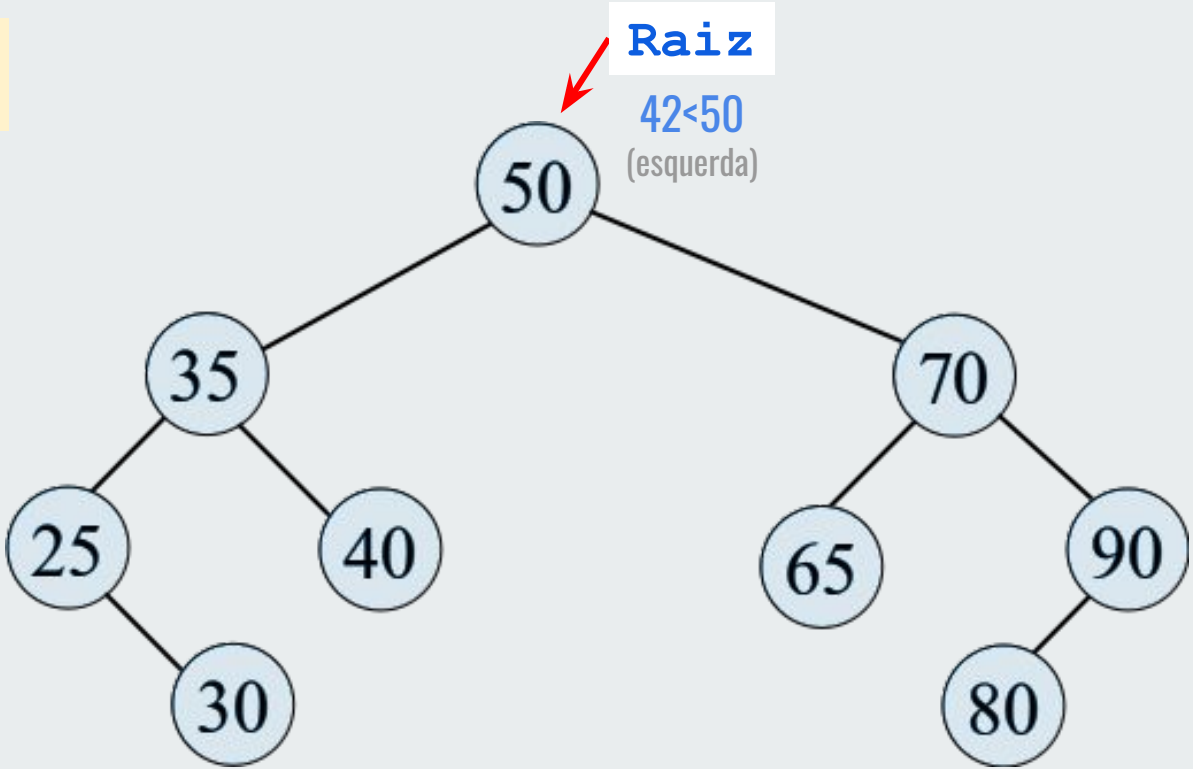


Inserção: O novo nó é inserido no lugar de uma subárvore vazia.

Inserir a chave: **42**

Pseudocódigo

```
pont_no inserir(pont_no raiz, int chave){
  // Se a árvore está vazia, cria um novo nó e retorna
  if (raiz == NULL) {
    pont_no novo = (pont_no)malloc(sizeof(no));
    novo->chave = chave;
    novo->esq = novo->dir = NULL;
    return novo;
  }
  // Se a chave é menor, insere na subárvore esquerda
  if (chave < raiz->chave)
    raiz->esq = inserir(raiz->esq, chave);
  // Se a chave é maior, insere na subárvore direita
  else if (chave > raiz->chave)
    raiz->dir = inserir(raiz->dir, chave);
return raiz; // Retorna a raiz (não alterada)
}
```

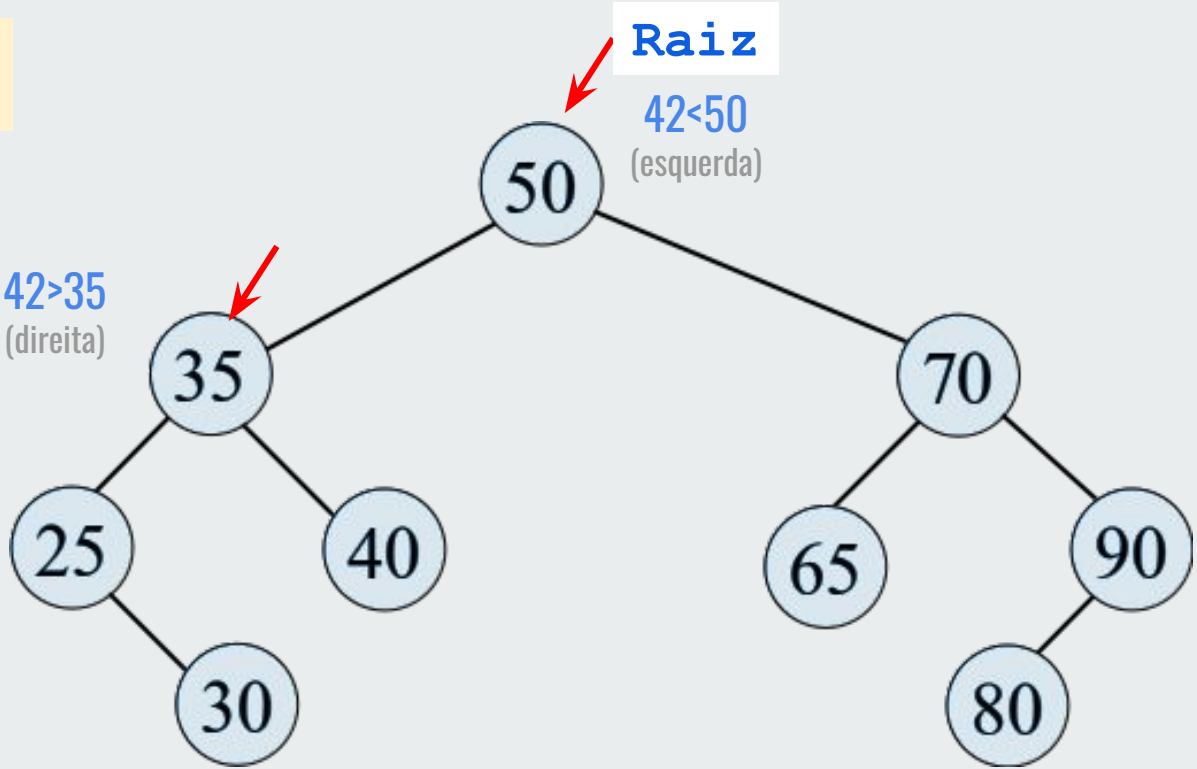


Inserção: O novo nó é inserido no lugar de uma subárvore vazia.

Inserir a chave: **42**

Pseudocódigo

```
pont_no inserir(pont_no raiz, int chave){
  // Se a árvore está vazia, cria um novo nó e retorna
  if (raiz == NULL) {
    pont_no novo = (pont_no)malloc(sizeof(no));
    novo->chave = chave;
    novo->esq = novo->dir = NULL;
    return novo;
  }
  // Se a chave é menor, insere na subárvore esquerda
  if (chave < raiz->chave)
    raiz->esq = inserir(raiz->esq, chave);
  // Se a chave é maior, insere na subárvore direita
  else if (chave > raiz->chave)
    raiz->dir = inserir(raiz->dir, chave);
return raiz; // Retorna a raiz (não alterada)
}
```

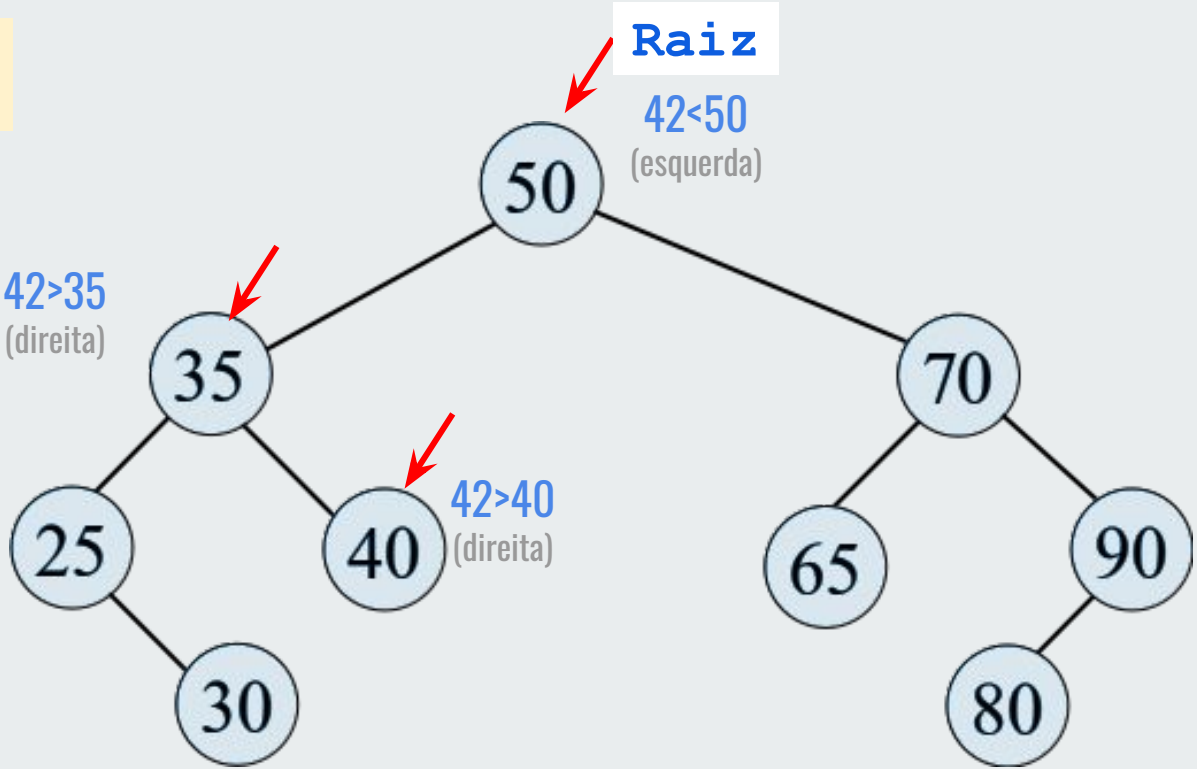


Inserção: O novo nó é inserido no lugar de uma subárvore vazia.

Inserir a chave: **42**

Pseudocódigo

```
pont_no inserir(pont_no raiz, int chave){
  // Se a árvore está vazia, cria um novo nó e retorna
  if (raiz == NULL) {
    pont_no novo = (pont_no)malloc(sizeof(no));
    novo->chave = chave;
    novo->esq = novo->dir = NULL;
    return novo;
  }
  // Se a chave é menor, insere na subárvore esquerda
  if (chave < raiz->chave)
    raiz->esq = inserir(raiz->esq, chave);
  // Se a chave é maior, insere na subárvore direita
  else if (chave > raiz->chave)
    raiz->dir = inserir(raiz->dir, chave);
return raiz; // Retorna a raiz (não alterada)
}
```

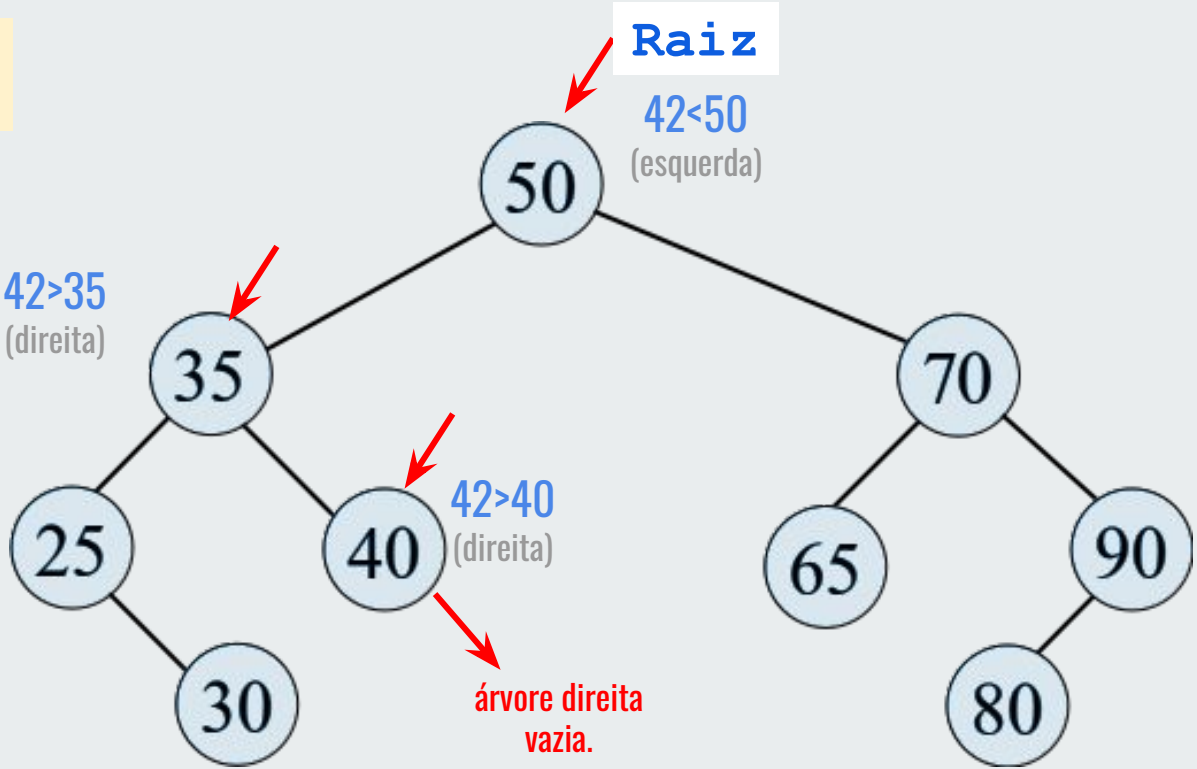


Inserção: O novo nó é inserido no lugar de uma subárvore vazia.

Inserir a chave: **42**

Pseudocódigo

```
pont_no inserir(pont_no raiz, int chave){
  // Se a árvore está vazia, cria um novo nó e retorna
  if (raiz == NULL) {
    pont_no novo = (pont_no)malloc(sizeof(no));
    novo->chave = chave;
    novo->esq = novo->dir = NULL;
    return novo;
  }
  // Se a chave é menor, insere na subárvore esquerda
  if (chave < raiz->chave)
    raiz->esq = inserir(raiz->esq, chave);
  // Se a chave é maior, insere na subárvore direita
  else if (chave > raiz->chave)
    raiz->dir = inserir(raiz->dir, chave);
return raiz; // Retorna a raiz (não alterada)
}
```

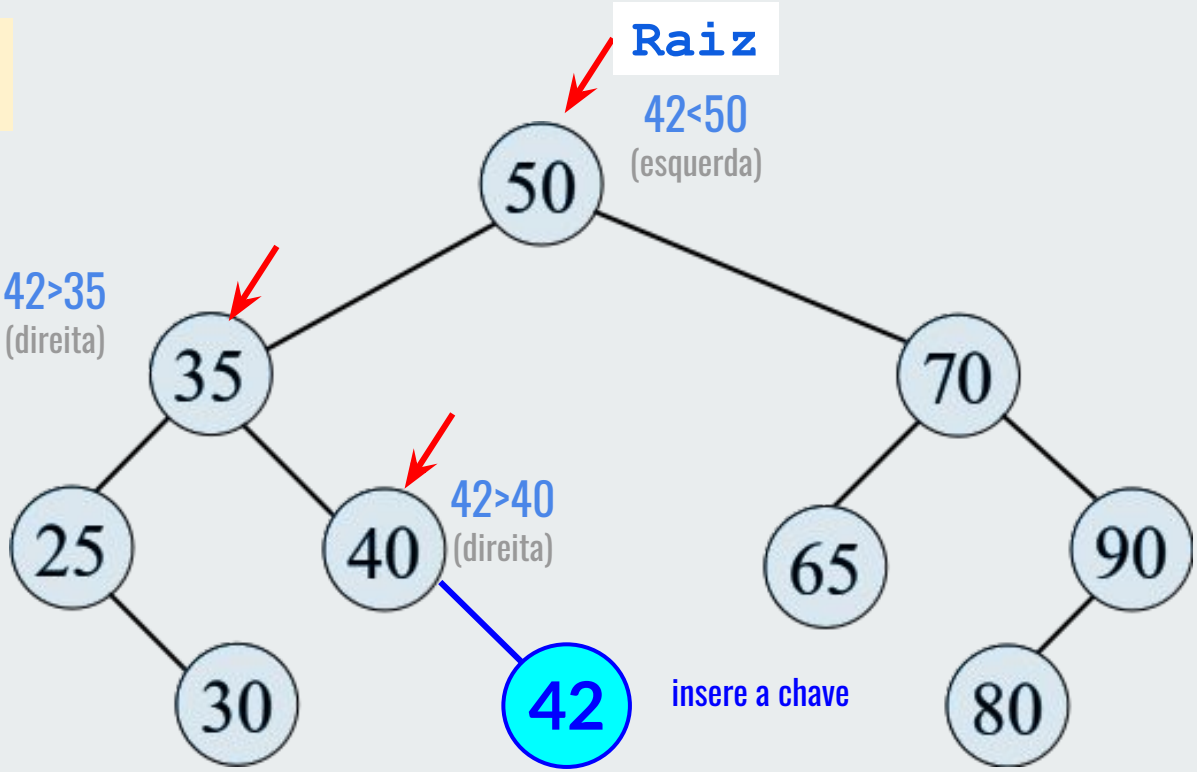


Inserção: O novo nó é inserido no lugar de uma subárvore vazia.

Inserir a chave: **42**

Pseudocódigo

```
pont_no inserir(pont_no raiz, int chave){
  // Se a árvore está vazia, cria um novo nó e retorna
  if (raiz == NULL) {
     pont_no novo = (pont_no)malloc(sizeof(no));
     novo->chave = chave;
     novo->esq = novo->dir = NULL;
     return novo;
  }
   // Se a chave é menor, insere na subárvore esquerda
  if (chave < raiz->chave)
     raiz->esq = inserir(raiz->esq, chave);
  // Se a chave é maior, insere na subárvore direita
  else if (chave > raiz->chave)
     raiz->dir = inserir(raiz->dir, chave);
return raiz; // Retorna a raiz (não alterada)
}
```



Pseudocódigo

```
struct nodo{
  int chave;
  struct nodo *esq;
  struct nodo *dir; };
typedef struct nodo no;
}
typedef no *pont_no;
```

Observação: se chave é igual a algum valor já existente, o código ignora a inserção (não faz nada)

Pseudocódigo

```
pont_no inserir(pont_no raiz, int chave){
  // Se a árvore está vazia, cria um novo nó e retorna
  if (raiz == NULL) {
    pont_no novo = (pont_no)malloc(sizeof(no));
    novo->chave = chave;
    novo->esq = novo->dir = NULL;
    return novo;
  }
  // Se a chave é menor, insere na subárvore esquerda
  if (chave < raiz->chave)
    raiz->esq = inserir(raiz->esq, chave);
  // Se a chave é maior, insere na subárvore direita
  else if (chave > raiz->chave)
     raiz->dir = inserir(raiz->dir, chave);
return raiz; // Retorna a raiz (não alterada)
}
```

COMPLEXIDADE??

Operações Básicas

1. BUSCA
2. INSERÇÃO
3. REMOÇÃO

INSERÇÃO

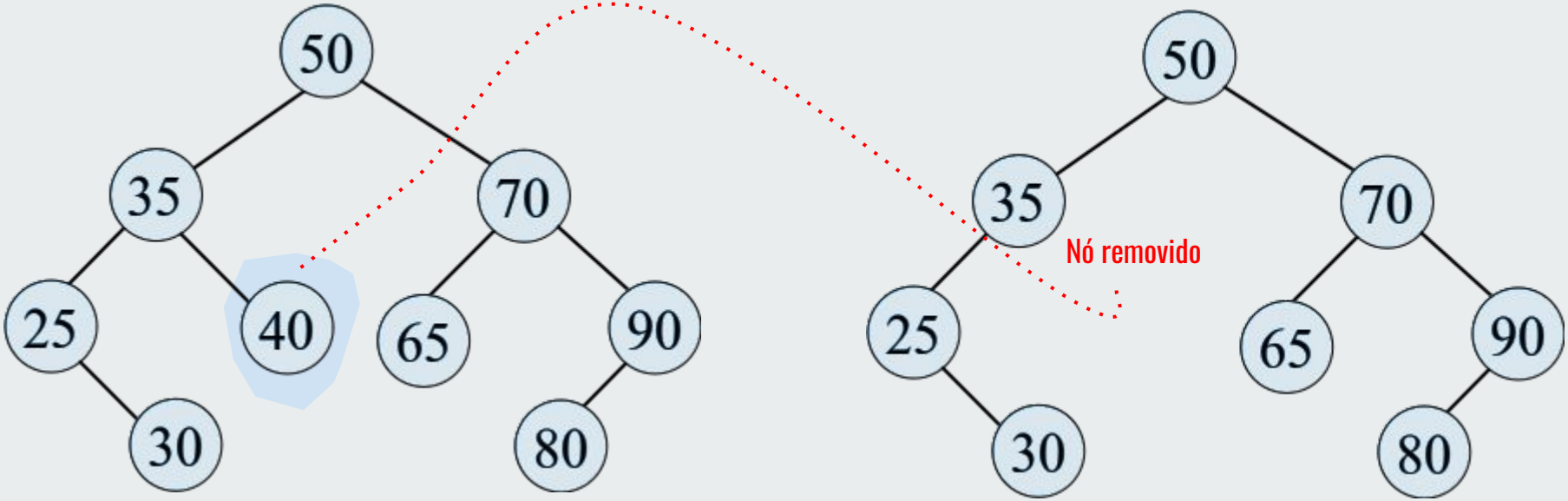
Remover um nó existente da árvore

Temos 3 casos possíveis:

1. *Deseja-se remover uma chave que está em uma folha.*
2. *A chave removida não é uma folha, mas possui uma subárvore vazia.*
3. *A chave removida não é uma folha e possui duas subárvores não vazias.*

CASO 1. Deseja-se remover uma chave que está em uma folha.

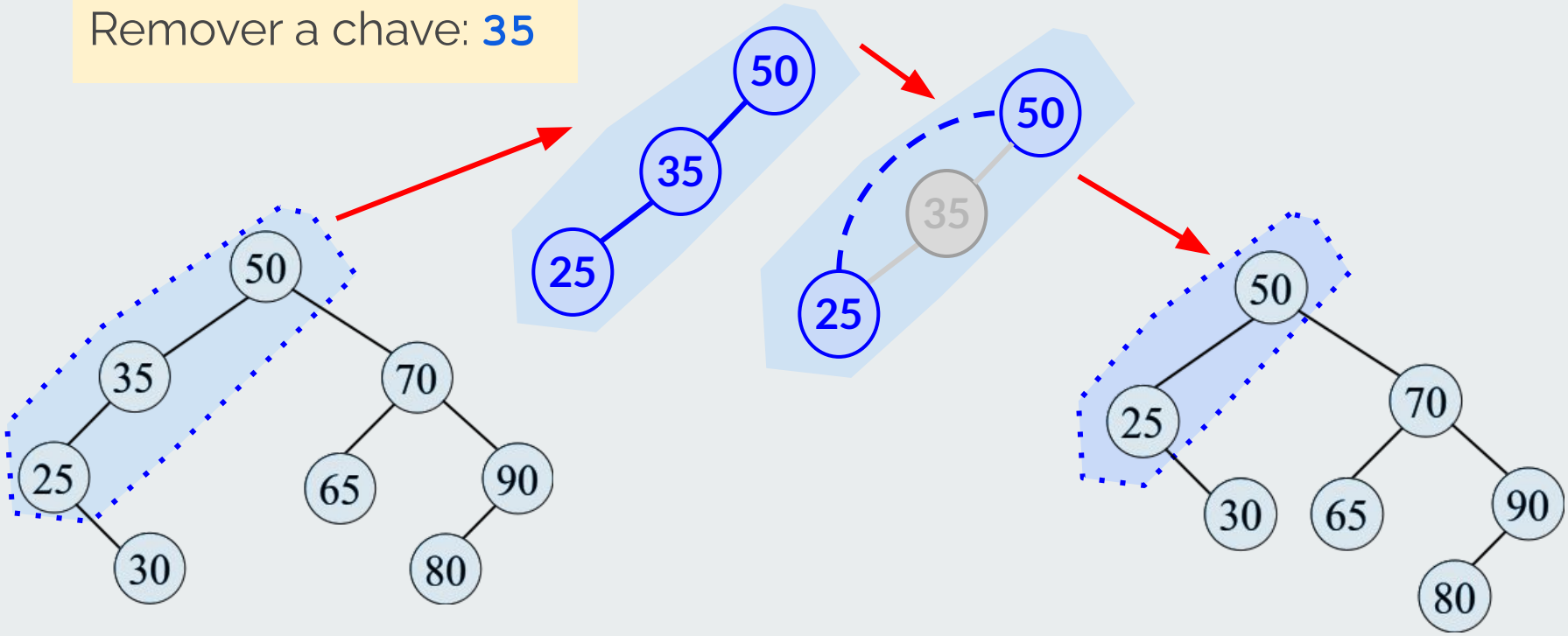
Remover a chave: **40**



Removemos o nó e ajustamos o ponteiro do pai para **NULL**.

CASO 2. A chave removida possui uma subárvore vazia.

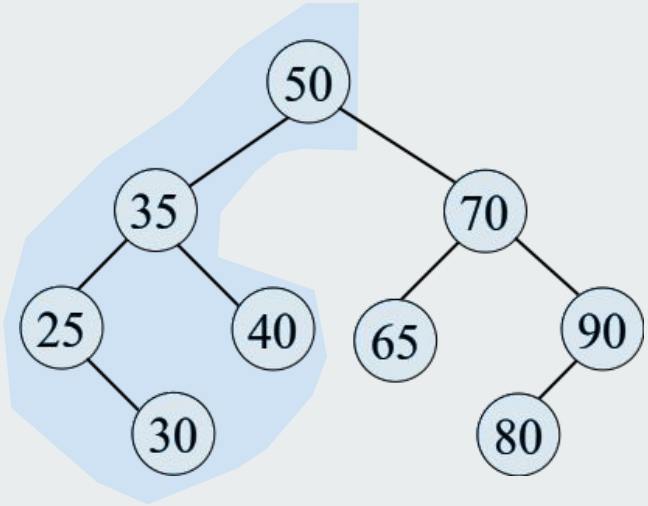
Remover a chave: **35**

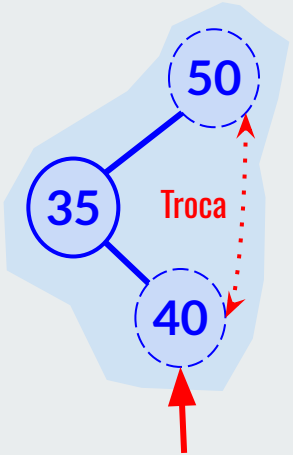


O filho do nó a ser removido substitui o nó.
Ligamos o pai do nó diretamente ao seu único filho.

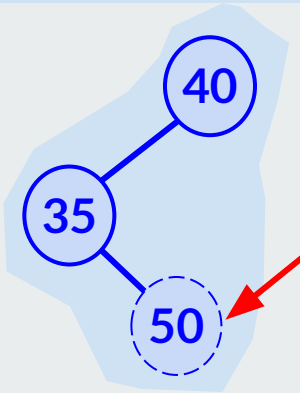
CASO 3. A chave removida possui duas subárvores não vazias.

Remover a chave: **50**

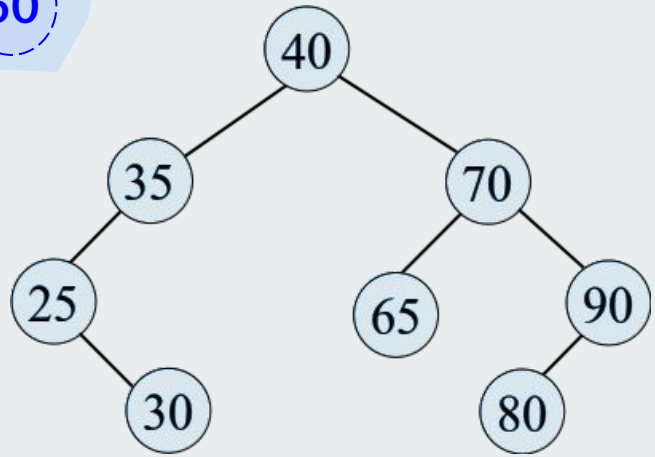




Maior elemento da subárvore esquerda



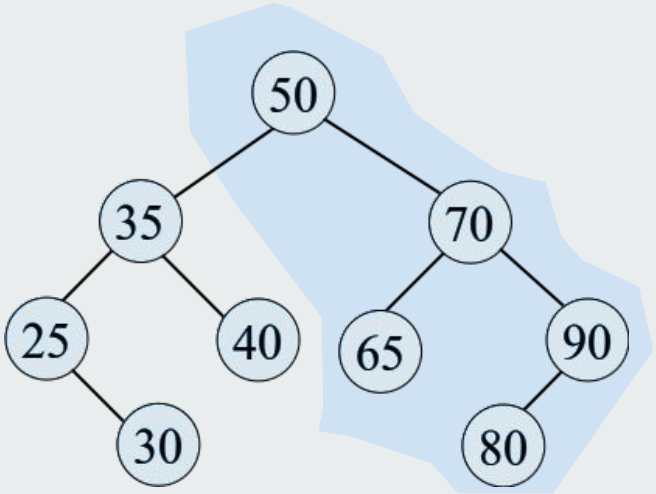
Virou uma folha (CASO 1)

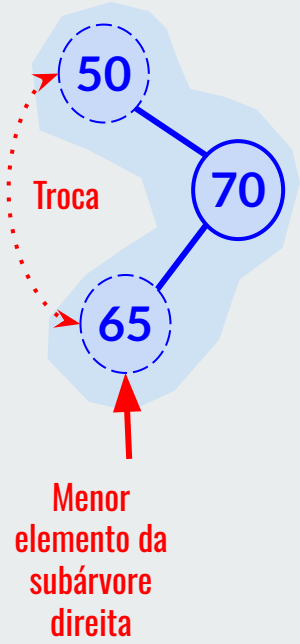


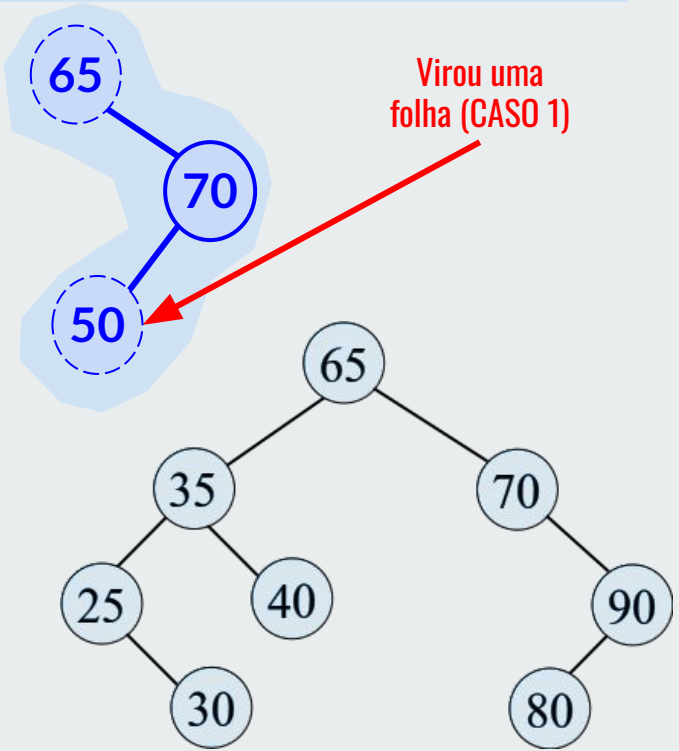
Substitui o elemento a ser removido pelo ***maior elemento da subárvore esquerda*** ou o menor da subárvore direita. Daí, ***reduzimos ao caso 1*** ou caso 2.

CASO 3. A chave removida possui duas subárvores não vazias.

Remover a chave: **50**







Substitui o elemento a ser removido pelo maior elemento da subárvore esquerda ou o ***menor da subárvore direita***. Daí, ***reduzimos ao caso 1*** ou caso 2.

Operações Básicas

1. BUSCA
2. INSERÇÃO
3. REMOÇÃO

COMPLEXIDADE?

Remover um nó existente da árvore

Temos 3 casos possíveis:

1. *Deseja-se remover uma chave que está em uma folha.*
2. *A chave removida não é uma folha, mas possui uma subárvore vazia.*
3. *A chave removida não é uma folha e possui duas subárvores não vazias.*

Vamos programar?



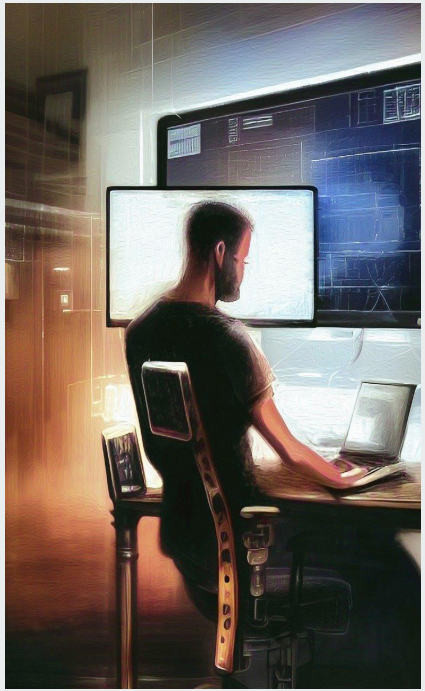
1. Implemente o procedimento de busca de uma chave em árvore binária .

2. Implemente a inserção de um nó em árvore binária.

3. Implemente a remoção de um nó em árvore binária.

4. Dada uma lista com **n** chaves, construir a árvore de busca binária de altura mínima utilizando o algoritmo de inserção. Qual a complexidade do seu método?





Dúvidas?!

